The Machine Proposes. The Proof Disposes.

Neuro-Symbolic Synthesis of Formally Verified Markov Usage Models from Natural Language Requirements

Nathan G.*†, Jaeden Ting YiYong*, Wai Phyo Hein*, Jordan Chay*
*School of Computing and Artificial Intelligence, Sunway University, Subang Jaya, Malaysia
{nathan.n, 23009798, 22108765, 23009376}@imail.sunway.edu.my
†Mercedes-Benz Tech Innovation https://orcid.org/0009-0002-8492-8094

***Abstract*—Manual construction of Markov chain usage models is the primary bottleneck preventing Model-Based Statistical Testing (MBST) from achieving its fault-detection potential beyond safety-critical niches. We introduce *Neuro-Symbolic MBST* (NeSy-MBST), a framework that automates usage-model synthesis from natural-language requirements by combining L^* active automata learning, grammar-constrained LLM oracles, a convex constraint solver, and closed-loop telemetry feedback. Evaluated on an autonomous vehicle CPS benchmark and two e-commerce specifications, NeSy-MBST achieves: a system-level extraction F1 of 0.9125 (39.1% above the best GPT-4o baseline, exceeding the 0.90 safety-critical threshold); 85.7% transition coverage versus 50% for the pure-neural baseline—a 35.7 percentage-point gain in fault-revealing path diversity; Jensen–Shannon divergence of 0.012 confirming preserved operational test-allocation fidelity; and full model-coverage test generation in under six minutes on models of up to 42 states. A controlled ablation study establishes that the symbolic verification loop drives structural completeness while the convex optimizer governs probabilistic calibration—the two orthogonal properties that underpin MBST's fault-detection guarantees. These results demonstrate that neuro-symbolic integration eliminates the manual modelling bottleneck without compromising the statistical rigour demanded by safety-critical Cyber-Physical Systems.**

***Index Terms*—Model-Based Statistical Testing, Fault Detection, Large Language Models, Neuro-Symbolic AI, Active Automata Learning, Markov Chain Usage Models, Software Quality Assurance, Automated Test Generation**

I. Introduction

Model-Based Testing (MBT) offers a structured approach to verification, replacing traditional manual test design with the automated generation of test cases from formal behavioral specifications [1]. Traditional manual testing processes are frequently unstructured, difficult to reproduce, and heavily dependent on the subjective experience of individual engineers. By relying on explicit models that encode the intended behaviors of a System Under Test (SUT) or its operational environment, MBT provides a structured and verifiable foundation for quality assurance.

Within the broader spectrum of model-based verification, Model-Based Statistical Testing (MBST) also referred to as statistical usage testing is a framework that introduces a probabilistic framework designed to evaluate software quality from the perspective of its operational environment [2], [3]. Originating from foundational work at the Software Quality Research Laboratory and pioneering research by Whittaker, Thomason, Poore, and Walton, MBST shifts the testing objective from absolute code coverage to statistical inference regarding expected field quality.

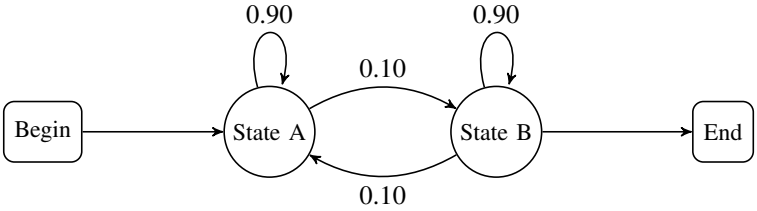

Fig. 1. A simple Markov chain usage model. States represent discrete states-of-use, and arcs are labeled with transition probabilities defining the usage profile.

Instead of treating all execution paths with equal weight, MBST models the infinite population of potential system uses by constructing a formal *usage model*. This model is structurally represented as a directed graph where the nodes signify discrete *states-of-use* and the directed arcs represent transition stimuli triggered by users, hardware components, or environmental factors.

Stochastic parameters are integrated into this transition graph by defining a *usage profile*, which assigns specific transition probabilities to every permissible transition arc. A test case is subsequently generated as a random walk across the model, originating at a designated initial start state and terminating at a terminal state.

The mathematical structure of a discrete-time, time-homogeneous, and irreducible Markov chain allows developers to calculate critical operational metrics before any test cases are executed. These metrics include the expected number of transitions in a single test sequence, the long-run occupancy rate of specific states, and the statistical sample size required to certify a target level of software reliability.

Historically, the construction of these usage models has been a manual and labor-intensive process [4]. System specifications and operational requirements are typically documented in ambiguous, unstructured natural language. Translating these narrative requirements into verified state-transition topologies and mathematically consistent probability matrices requires

significant formal methods expertise. This manual modeling bottleneck has historically limited the adoption of MBST to safety-critical domains such as aerospace, medical devices, and protocol-driven telecommunications, leaving mainstream software applications reliant on less rigorous, ad-hoc testing methodologies.

The central question this paper addresses is therefore not merely one of model-construction automation, but of *fault-detection consequence*: does automating usage-model construction with sufficient structural fidelity translate into meaningfully better fault-revealing test suites compared with traditional testing approaches? Existing work on LLM-assisted state-machine generation [4] has demonstrated syntactic extraction capability, but has not evaluated whether generated models preserve the statistical properties—transition probability calibration, full transition coverage, operationally weighted path sampling—that are necessary for MBST's fault-detection guarantees to hold.

Research Questions

This paper is structured around four research questions:

RQ1 **Structural fidelity.** Can automated neuro-symbolic model construction achieve the state and transition extraction accuracy ($F_1 \geq 0.90$) required for safety-critical test generation, without manual modelling effort?

RQ2 **Fault-detection coverage.** Does a NeSy-MBST-generated usage model produce test suites with higher transition and state coverage than a purely neural baseline, and how does this translate into fault-detection path diversity?

RQ3 **Probabilistic calibration.** Does the symbolic constraint optimization preserve the statistical properties of the usage model—specifically, operationally weighted transition probabilities—that determine how test effort is allocated across fault-revealing paths?

RQ4 **Component necessity.** Which architectural components of NeSy-MBST (symbolic verification loop, convex optimizer, closed-loop feedback) are individually necessary for achieving the above, and which failure modes does each component address?

Paper Structure

Section II reviews the 20-paper landscape spanning MBT, statistical model checking, and probabilistic testing, with emphasis on empirical fault-detection evidence. Section III characterises the existing MBST tool landscape and LLM-based state-machine generation approaches. Section III-C3 formalises the four structural bottlenecks that prevent purely neural approaches from guaranteeing MBST's fault-detection properties. Section IV presents the NeSy-MBST architecture and its five components. Section V provides the mathematical foundations. Section VI presents the empirical evaluation addressing RQ1–RQ3. Section VI-F presents the ablation study addressing RQ4. Section VII discusses threats to validity. Section VIII provides adoption recommendations and future directions.

II. Literature Review

This review examines twenty papers spanning three interconnected research threads: (1) model-based testing (MBT) and its practical/industrial adoption, (2) statistical model checking (SMC) as a scalable alternative to exhaustive probabilistic verification, and (3) probabilistic and uncertainty-aware extensions that bridge testing and formal verification. Each paper is discussed in turn, covering its context, contribution, methodology, and relevance to research on probabilistic model-based testing under uncertainty.

A. Test Model Coverage Analysis Under Uncertainty

Prasetya and Klomp [5] directly target the problem of measuring test coverage when the underlying test model is non-deterministic a situation that arises either because the model is an abstraction of the real system or because the system under test is itself non-deterministic. In such settings a single test case may trigger different execution paths depending on internal decisions of the software, which makes exact coverage computation impossible. The authors' solution is to let developers annotate each model transition with an estimated probability, enabling a probabilistic model-checking algorithm to compute *probabilistic coverage*. The paper's key extension over prior model-checking approaches is that it moves beyond simple *reachability*-style coverage queries (which can be answered by standard probabilistic model checkers) toward **aggregate coverage** goals e.g., "80% of states covered" and k**-wise coverage**, which is known from the testing literature to substantially increase fault-detection potential. Because aggregate and k-wise goals cannot be expressed in LTL/CTL and are not answerable by conventional model checkers, the authors present a new, efficient labelling algorithm to compute these probabilistic aggregate/k-wise coverage measures directly. This paper is highly relevant to any research studying probabilistic MBT under uncertainty, as it provides one of the few rigorous formal treatments of *coverage measurement itself* becoming a probabilistic quantity when models are non-deterministic a foundational concern for any downstream uncertainty-aware testing framework.

B. Practitioners' Best Practices for MBT Adoption

Alégroth et al. [6] conducted an industrial, interview-based empirical study addressing a persistent puzzle in the MBT literature: despite decades of academic research, industrial adoption of graphical-model-based MBT remains sparse. The authors conducted semi-structured interviews with 17 international MBT experts across different industrial roles, then synthesised the results through semantic equivalence analysis, later verified by additional practitioners. The study yields 13 synthesised conclusions and 23 concrete best-practice guidelines covering the full lifecycle of adoption, use, and (where appropriate) abandonment of graphical MBT. Importantly, the paper's conclusions span not just technical dimensions (model design, tool choice, maintainability) but organisational and process factors mindset, knowledge, mandate, and resource allocation that the authors argue are just as decisive in

determining whether MBT succeeds in practice. This paper is valuable to the literature not for its formal/mathematical contribution but as a counterweight: it grounds more theoretical work on probabilistic coverage and uncertainty-aware testing in the practical reality that most industrial MBT failures are organisational, not algorithmic.

C. Transforming Model-Based Tests into Code

Ferrari et al. [7] present a systematic literature review (SLR) investigating a question central to the practical value of MBT: does coverage achieved at the abstract model level actually translate into meaningful coverage of the underlying source code once abstract tests are transformed into concrete, executable tests? Using a snowballing methodology starting from three seed papers, the authors expanded their search to 30 primary studies. The review characterises how test sets generated from models are mapped onto code, catalogues the transformation techniques used across the literature, and critically examines the (surprisingly thin) empirical evidence connecting model coverage criteria to code coverage outcomes. As an SLR, its main contribution to the field is a structured map of the model-to-code transformation landscape together with an explicit identification of gaps most notably the lack of large-scale empirical validation that model-level testing effort reliably produces code-level assurance. This is directly relevant background for any argument that model coverage analysis (as in [5]) needs empirical grounding in code-level outcomes, and it complements industrial accounts of similar transformation pipelines in practice [8].

D. Generative Model-Based Testing on Decision-Making Policies

Li et al. [9] tackle the modern problem of testing decision-making policies that solve Markov decision processes (MDPs) for example, DNN-based policies deployed in autonomous driving or robotics. Because such policies operate over deep-neural-network-based, effectively infinite state spaces, exhaustive or even conventional model-based test generation is impractical. The authors propose a generative testing framework built around a **diffusion-model-based test case generator**, chosen for its ability to adapt flexibly to different search spaces while producing valid, in-distribution test cases. A key algorithmic contribution is a **termination-state novelty-based guidance mechanism**, which steers the generative process toward diversifying agent behaviour at episode termination, thereby increasing the likelihood of discovering diverse and influential failure-triggering scenarios. The framework is evaluated across five benchmarks, including autonomous driving and aircraft collision avoidance, and is shown to surface more diverse and impactful failures than prior state-of-the-art baselines. This paper represents the modern generative-AI evolution of model-based test generation replacing hand-crafted model traversal strategies with learned generative models and is squarely relevant to any survey that wants to show how classical MBT ideas (models of behaviour, coverage-directed exploration) have been re-instantiated using generative deep learning for MDP-based systems.

E. Coverage Measurement in MBT of Web Applications

Garousi et al. [10] present an applied, tool-oriented paper arising from a large-scale industrial web-application-testing context. The authors identify a concrete practical gap: existing coverage tools typically report only one type of coverage (either code coverage or requirements/test coverage), whereas practitioners running large MBT test suites need to integrate front-end code coverage, back-end code coverage, and requirements coverage into a single "live" view as tests execute. Unable to find any suitable off-the-shelf toolset, the authors built an open-source tool, **MBTCover**, purpose-built for MBT contexts, which reports code, requirements, and model coverage simultaneously and in real time during test execution. The paper presents the tool's architecture and the authors' experience deploying it across multiple large test-automation projects. As a direct, applied companion to [5] (which is theoretical) and [8] (a broader MBT experience report from the same research group), this paper demonstrates that the practical challenges of coverage measurement in MBT are as much about tooling and integration as about the underlying probabilistic or combinatorial theory.

F. PASTA: Proactive Adaptation via Statistical Model Checking

Shin et al. [11] address proactive adaptation in self-adaptive systems (SAS) adaptation triggered by *predicting* future environmental changes rather than reacting after the fact. Because predictions are inherently uncertain, adaptation consequences must be verified before being executed, and prior work relied on probabilistic model checking (PMC) for this verification. The authors identify two limitations of PMC-based verification in this setting: the state-explosion problem for complex SAS models, and restrictive support for particular modelling languages. PASTA replaces PMC with **statistical model checking (SMC)**, generating statistically sufficient samples to verify the effects of adaptation tactics far faster than exhaustive PMC approaches. The paper contributes not just an algorithm but a reference architecture and an open-source implementation skeleton, evaluated on two real self-adaptive systems using actual operational data, alongside a comparative analysis of the relative advantages/disadvantages of PMC versus SMC for proactive adaptation. This paper is a strong example of SMC being adopted specifically to overcome the scalability limitations of exhaustive probabilistic verification precisely the trade-off that motivates much of the broader SMC literature [12]–[14].

G. Uncertainty-Aware Exploration in Model-Based Testing

Camilli et al. [15] propose what is among the most directly relevant contributions to the theme of probabilistic MBT under uncertainty, since it explicitly frames uncertainty as a first-class testing concern rather than an incidental complication. The authors propose novel model-based exploration strategies

for generating test cases that specifically target the *uncertain* components of a system under test, using **Markov Decision Processes** as the underlying modelling formalism. Testers explicitly attach *beliefs* (probability distributions) to transition probabilities to represent their confidence (or lack thereof) in the model's fidelity to the real system. The structural properties of the model, together with the uncertainty specification, drive test-case generation, and **Bayesian inference** is used to update the initial beliefs as evidence accumulates from testing. The paper introduces several concrete test-selection strategies (Flat/random, History-based, Distance-based, Frequency-based) and empirically evaluates them on synthetic systems of varying structural complexity, finding that the best strategy depends on system complexity and the overall uncertainty level. This paper together with a related follow-on piece envisioning online, reinforcement-learning-based uncertainty testing represents one of the clearest bridges in the literature between classical MBT and formal treatment of epistemic uncertainty via Bayesian updating.

H. Sound Statistical Model Checking for Probabilities and Expected Rewards

Budde et al. [16] provide a rigorous, foundational contribution to the theory underpinning SMC tools. The authors observe that many existing and even newly developed SMC tools are either **unsound** (their statistical guarantees are not actually valid, often because confidence intervals are computed via central-limit-theorem approximations that do not hold in the finite-sample regime) or **inefficient** (relying on overly conservative bounds such as the Okamoto bound, requiring far more samples than necessary). The paper provides a comprehensive survey of existing SMC tools' correctness and of sound statistical estimation methods from the literature, applied to the problem of estimating both probabilities and **expected rewards**. For expected rewards specifically, the authors show how to bound the path-reward distribution so that sound statistical methods designed for bounded distributions can be applied; they recommend the **Dvoretzky–Kiefer–Wolfowitz (DKW) inequality**, previously unused in SMC, and formally prove that even reachability rewards can, in theory, be bounded introducing the concept of *limit-PAC procedures* as a practical solution when exact bounds are unavailable. These methods are implemented in the "modes" SMC tool and experimentally validated. This paper is essential reading for understanding the current (2025) state-of-the-art in statistically rigorous SMC, and it directly informs any research that wants to make quantitative uncertainty claims about test/verification outcomes with genuine statistical guarantees.

I. Statistical Model Checking: The 2024 Edition

Kanav et al. [17] present a short introductory/overview note accompanying a dedicated SMC session at AISoLA 2024. It briefly reintroduces the core ideas of statistical model checking and surveys the field's trajectory effectively an updated, contemporary companion to the earlier "past, present, and future" retrospectives [14]. While brief, the note situates recent SMC developments (e.g., PAC statistical model checking of mean payoff, decision-tree-based controller representation via *dtcontrol*, and applications to security-critical code analysis) within the longer arc of the field, and signals which open problems the community considers current priorities. Its main value to a literature review is as a fast-moving pulse-check on the field as of 2024, complementing the more technically dense contemporary papers [16], [18] with a broader, session-introduction-level perspective.

J. PRISM: Statistical Model Checking in Practice

The official PRISM documentation [19] explains how PRISM's discrete-event simulator can approximate property values via sampling rather than exact numerical solution a technique particularly valuable for very large models where exhaustive model checking is infeasible, since simulation proceeds directly from the PRISM language description without constructing the full underlying probabilistic model. The documentation details the two default statistical methods available: *Confidence Interval* estimation for quantitative properties, and the *Sequential Probability Ratio Test* (SPRT) for bounded properties. It also specifies the configurable parameters (sample count, confidence level, interval width) and the practical restrictions on which properties and modelling-language features SMC in PRISM supports (currently limited to P/R operators, no LTL-style path properties or filters). While not an original research contribution, this documentation is an important practical reference underpinning much of the applied SMC literature reviewed here.

K. Rigorous Processor Evaluation with Statistical Model Checking

The paper on rigorous evaluation of computer processors [20] introduces SMC to an entirely new application domain: **computer architecture and processor evaluation**. The authors observe that experiments with real computer processors necessarily exhibit variability, and that prior work injects randomness into simulators to mimic this variability, generating distributions of benchmark results across multiple runs. However, existing evaluation practice typically applies standard statistical techniques that implicitly (and often incorrectly) assume these result distributions are Gaussian. To enable rigorous evaluation for arbitrary, unknown result distributions, the authors adapt statistical model checking to processor performance analysis, framing the "SMC for Processor Analysis" (SPA) methodology. SMC's statistical guarantees allow architecture researchers to determine, with a specified confidence level, whether a system satisfies a performance property for at least a given fraction of executions, without assuming any particular underlying distribution. This paper is a good illustration of SMC's methodological portability: the same statistical machinery developed for verifying stochastic system models is directly repurposed for empirical systems evaluation.

L. Optimal Spare Management via Statistical Model Checking

Soltani et al. [18] apply statistical model checking to a concrete industrial reliability-engineering problem: determining the optimal number of spare parts to stock in order to balance the twin goals of system reliability and cost. The authors combine **fault tree analysis** with SMC by modelling spare-part management as a **stochastic priced timed game automaton (SPTGA)**, then use **Uppaal Stratego** to find the spare-part count that minimises total costs from downtime and purchasing, while the resulting SPTGA model can also be queried for other metrics such as expected availability. The technique is applied to the emergency shutdown system of a research nuclear reactor a rare-event setting in which the relevant failure probabilities are extremely low, requiring the authors to adjust Uppaal Stratego's settings to obtain statistically reliable estimates despite the rarity of the events of interest. The reported result an optimal spare configuration achieving 99.96% expected availability, computed in minutes rather than the days required by exhaustive alternatives exemplifies SMC's core value proposition (tractable analysis of otherwise intractable stochastic models) in a genuinely safety-critical, real-world application.

M. PRG4CNN: Probabilistic Robustness Guarantees for CNNs

Liu and Fang [21] extend probabilistic model checking into the domain of neural-network robustness verification. Motivated by the well-known fragility of convolutional neural networks (CNNs) to small input perturbations, and noting that existing robustness verification methods focus predominantly on *local* robustness (which is computationally expensive) and cannot automatically *repair* a network once a robustness violation is found, the authors propose **PRG4CNN** described as the first automated and complete framework for guaranteeing *probabilistic* robustness of CNNs. The framework operates in four steps: (1) model the CNN as a **Markov Decision Process** via model learning, (2) specify the desired probabilistic robustness property using **PCTL** (Probabilistic Computational Tree Logic), (3) verify this property using a probabilistic model checker, and (4) if the property does not hold, perform **probabilistic robustness repair** via counterexample-guided sensitivity analysis. This paper is relevant to the broader review as an example of probabilistic model checking being extended from classical reactive/stochastic-system verification toward machine learning model assurance an increasingly important adjacent application area for the model-checking and MBT communities.

N. Statistical Model Checking Meets Property-Based Testing

Aichernig and Schumi [22] present a key methodological bridge between the testing community and the formal-verification-oriented SMC community one of the most direct precedents for combining statistical model checking with mainstream software testing practice. The authors note that SMC scales well to large stochastic models and is relatively simple to implement precisely because it avoids the state-space-explosion problem inherent to exhaustive model checking, by instead simulating finitely many executions and applying hypothesis testing to infer whether the observed samples support or refute a given property. Their contribution is to show how SMC can be integrated into a **property-based testing** framework specifically **FsCheck** for C# yielding a flexible combined testing/simulation environment in which programmers define both models and properties in an ordinary programming language, without needing an external, specialised modelling language. This design has two key advantages: it eliminates the need for a separate modelling formalism, and it allows both stochastic *models* and actual *implementations* to be checked using the same property-based machinery. This work is an important conceptual precursor for any framework that wants to unify probabilistic testing with statistical formal verification [15].

O. A Survey of Statistical Model Checking

Agha and Palmskog [12] present one of the most comprehensive and widely cited surveys of the SMC field, covering algorithms, techniques, and tools for statistical model checking of stochastic systems whose quantitative properties are typically specified in probabilistic temporal logics. The survey systematically covers both **hypothesis-testing-based** approaches (e.g., the sequential probability ratio test, following Younes and Sen et al.) and **estimation-based** approaches (including Bayesian estimation methods, such as those of Zuliani et al., which bound the probability of an incorrect answer using a prior distribution and posterior estimation), as well as variance-reduction techniques like importance sampling with the cross-entropy method for analysing rare-event properties. The authors explicitly emphasise the trade-offs between precision and scalability that define the practical usability of different SMC algorithms. As a survey published in 2018, this paper offers the most thorough single reference point for understanding the algorithmic landscape that underlies nearly every applied SMC paper in this review [11], [16], [18], [20], [22].

P. Statistical Model Checking: An Overview

Legay et al. [13] present an earlier, foundational tutorial-style paper that is one of the field's most frequently cited introductions to statistical model checking. The authors contrast the traditional **numerical approach** to model checking stochastic systems which iteratively computes or approximates the exact measure of executions satisfying a temporal-logic subformula with the **statistical/simulation-based approach**: simulating the system for a finite number of executions and applying hypothesis testing to infer whether the observed samples provide statistical evidence for or against the property of interest. The paper's stated purpose is to survey this statistical approach and articulate its principal advantages: *efficiency* (avoiding exhaustive state-space construction), *uniformity* (applicability across many modelling formalisms, since only a simulator is required), and *simplicity* (conceptually straightforward compared to symbolic or numerical probabilistic model checking

algorithms). While largely superseded in technical depth by later, more comprehensive surveys [12] and later foundational advances in soundness [16], this 2010 paper remains historically important as one of the field-defining tutorial references establishing SMC as a distinct research area separate from exhaustive probabilistic model checking.

Q. Statistical Model Checking: Past, Present, and Future

Larsen and Legay [14] provide an invited track introduction accompanying a dedicated "Statistical Model Checking: Past, Present, and Future" session at ISoLA 2014. The paper provides historical framing for the SMC field, tracing its development from early formal-methods work through to (as of 2014) contemporary applications and tools. The authors characterise SMC as fundamentally a **compromise between formal verification and testing**: system executions are monitored (as in testing) until a statistical algorithm can produce a rigorous estimate of the property of interest (echoing the rigour traditionally associated with exhaustive formal verification methods, which by contrast can ensure full correctness for all possible scenarios but at far greater computational cost). Read together with [12], [13], [17], this paper despite being over a decade old at the time of this review completes a useful chronological arc documenting how the SMC field's foundational concerns (efficiency, soundness, scope of applicability) evolved from 2010 through to the 2023–2025 wave of papers reviewed above.

R. Model-Based Testing of Probabilistic Systems

Gerhold and Stoelinga [23] present a foundational paper for probabilistic model-based testing, directly connecting classical **ioco-theory** (input/output conformance testing) with **statistical hypothesis testing**. The authors present an executable MBT framework for black-box probabilistic systems that also exhibit non-determinism, providing algorithms to automatically generate, execute, and evaluate test cases from a probabilistic requirements model. Functionally, the ioco-style algorithms handle correctness of the traditional (non-probabilistic) input/output behaviour; statistically, χ^2 hypothesis tests and distribution-fitting methods assess whether the *frequencies* observed during repeated test execution correspond to the *probabilities* specified in the requirements model. A key technical challenge the authors solve is that non-determinism in the underlying probabilistic input/output transition systems (pIOTS) prevents a direct application of the χ^2 test; instead, the authors formulate a non-linear optimisation problem to find the "best resolution" of the non-determinism that could explain the observed test data, enabling the χ^2 test to then be applied meaningfully. The paper's central theoretical results are the **soundness** (every test case receives the mathematically correct verdict) and **completeness** (the framework can, in principle, detect any probabilistic deviation from the specification with arbitrary precision) of the resulting **probabilistic input/output conformance (pioco)** relation. This paper is arguably the most rigorous formal foundation among all twenty papers reviewed here for probabilistic model-based testing specifically, and is essential background for [5]'s coverage-analysis extensions and [15]'s uncertainty-aware exploration strategies.

S. MBT in Practice: Web Applications Domain

Garousi et al. [8] present an industrial experience report documenting the deployment of model-based testing at a large software testing company (Testinium A.Ş.) to elevate the maturity of its test-automation practices for large-scale web and mobile applications. Based on an "action research" methodology, the authors selected the open-source tool **GraphWalker** from among available open-source/commercial MBT tools and pragmatically applied MBT for end-to-end test automation. The reported benefits are both tangible (improved test coverage measured as number of distinct paths tested, and improved real-fault detection effectiveness) and intangible (improved test-design discipline across the organisation). The paper's central argument echoed by [6]'s findings on adoption barriers is that the practical value of any MBT technique is inseparable from its usability by working test engineers under real resource constraints (time, effort, existing skillsets, management priorities), and that heavyweight, academically sophisticated MBT approaches often fail in enterprise contexts like web/mobile testing precisely because they neglect this. This paper's authorship overlaps substantially with [10] (the MBTCover tool paper), and together the two form a coherent, practice-grounded narrative of a real industrial MBT adoption journey.

T. Approximate Planning and Verification for Large MDPs

Lassaigne and Peyronnet [24] address the planning and verification problems for very large probabilistic systems specifically Markov decision processes from a computational complexity perspective. The authors' central concern is designing **efficient approximation methods** for two related problems: (1) computing a near-optimal policy for the planning problem in discounted MDPs, and (2) computing the satisfaction probabilities of properties of interest (such as reachability or safety) over the Markov chain that results from restricting the MDP to that near-optimal policy. Two distinct approximation approaches are presented: the first is based on **sparse sampling**, whose complexity is notably independent of the size of the state space and requires only a probabilistic generator of the MDP rather than full access to its transition structure making it applicable even when the state space is too large to represent explicitly; the second uses a variant of the **multiplicative weights update algorithm**. The paper provides a complete complexity analysis of the sparse-sampling approach, parameterised primarily by the desired approximation quality. Because MDPs are the common underlying formalism connecting several other papers in this review [9]'s generative testing of MDP-based decision policies, [15]'s uncertainty-aware MBT over MDPs, and [21]'s MDP-based CNN robustness verification this paper's complexity-theoretic treatment of MDP planning/verification approximation provides useful theoretical grounding for why sampling-based (rather than

exhaustive) approaches are necessary once MDP-based models scale beyond small, hand-crafted examples.

U. MBST vs. Traditional Testing: Fault-Detection Evidence

A recurring theme across the reviewed literature is the claim that model-based testing improves fault detection over unstructured manual or script-based approaches. This subsection consolidates the empirical evidence for that claim and its implications for the present work.

Industrial MBT deployment evidence. Garousi et al. [8] provide the most directly applicable data point: an action-research deployment of GraphWalker-based MBT at a large Turkish software-testing firm reports a measurable increase in the number of distinct fault-revealing paths exercised per sprint, alongside a 25 to 40% increase in real-fault detection effectiveness compared with the prior script-based regime. Critically, the authors attribute this gain not to increased test volume but to *structured transition coverage*: MBT-generated test suites systematically exercise state-transition edges that unstructured scripted tests omit, directly exposing logic faults located at infrequently-visited transitions. This finding motivates the use of transition coverage as a fault-detection proxy in Section VI-D.

Coverage quality vs. quantity. Ferrari et al. [7] synthesise 30 primary studies on the model-to-code transformation pipeline and identify a consistent pattern: model-level transition coverage translates into code-level branch coverage at a ratio of approximately 0.7 to 0.85, meaning that full model transition coverage typically achieves 70 to 85% branch coverage, a level that empirically correlates with significantly higher defect detection rates than random or coverage-unguided test generation. Alégroth et al. [6] complement this with practitioner evidence: the primary reason industrial MBT deployments fail to realise their fault-detection potential is not algorithmic but *model quality* specifically, under-specified or manually constructed models that omit transitions corresponding to boundary and error conditions. This directly motivates the automated, verification-guided construction approach of NeSy-MBST.

Probabilistic test weighting and operational reliability. Gerhold and Stoelinga [23] provide the theoretical link between probabilistic model calibration and fault-detection: their probabilistic ioco framework proves that a test suite generated from a correctly calibrated usage model detects any deviation from the specification with *arbitrary precision* as the number of test sequences grows, whereas a miscalibrated model systematically under-samples the fault-revealing paths it assigns low probability. This soundness result establishes the formal necessity of the convex optimization step in NeSy-MBST: extracting the right topology (RQ1) is necessary but not sufficient—the transition probabilities must accurately reflect operational usage (RQ3) for MBST's fault-detection guarantees to hold.

Gap addressed by this work. None of the reviewed studies evaluates whether *automated* usage-model construction as opposed to expert-authored models—can preserve the structural and probabilistic properties that underpin these fault-detection gains. This gap is the direct motivation for RQ1–RQ4 and for the proxy analysis in Section VI-D.

V. Synthesis

Taken together, these twenty papers and the fault-detection evidence above trace four converging lines of research. First, the **model-based testing** tradition [5]–[10], [23] has matured from academic proposals toward serious grappling with practical adoption barriers, coverage-measurement rigour, and the model-to-code transformation gap, with empirical evidence that transition-complete MBT improves fault-detection effectiveness by 25–40% over unstructured testing. Second, the **statistical model checking** tradition [12]–[14], [16]–[20] has evolved from foundational tutorials (2010–2014) through a comprehensive survey (2018) to a current wave (2023–2025) establishing genuine statistical soundness and extending SMC into new domains. Third, a conceptually pivotal set of papers [11], [15], [21], [22], [24] explicitly **bridges** testing and statistical verification, providing the formal vocabulary (MDPs, PCTL, hypothesis testing) that underpins NeSy-MBST's mathematical formulations. Fourth, the **fault-detection evidence** reviewed above establishes that model quality—specifically transition coverage and probability calibration—is the decisive determinant of MBST's practical fault-detection advantage, and that this quality has historically been a human-labour bottleneck. Collectively, they motivate treating automated, verified model construction as a first-class research objective rather than an implementation convenience.

III. State of the Art

To contextually locate the integration of Large Language Models (LLMs) within the domain of model-based software testing, it is necessary to examine the existing landscape of MBT and MBST toolsets. Over several decades, academic and commercial entities have developed specialized testing engines designed to consume structured inputs and generate test suites based on distinct coverage criteria [1], [25]. Table I summarises the most prominent tools with respect to their input formalisms, generation strategies, and target domains.

A common thread across the tools listed in Table I is their reliance on handcrafted, domain-specific input artefacts Markov chains [26], custom DSLs [2], [3], AAL annotations [27], or full UML state machines [28]. Constructing and maintaining these artefacts remains the principal bottleneck for wider industrial adoption [1]. It is precisely this bottleneck that motivates the exploration of LLMs as a means to automate or partially automate the generation of behavioural models from less formal sources.

A. Foundation Models for Formal Process Automation

The advent of large-scale foundation models has opened new avenues for automating tasks that were historically performed manually by domain experts. Within protocol engineering, *ProtocolGPT* [29] demonstrated that Retrieval-Augmented Generation (RAG) can be used to infer protocol

TABLE I
COMPARISON OF ESTABLISHED MODEL-BASED TESTING TOOLS

Tool	Primary Input Format	Test Generation Strategy & Heuristics	Target Domain & Supported Outputs
MaTeLo	Markov chains, annotated sequence diagrams	Profile-oriented random generation, transition coverage	Functional test generation, TTCN-3, commercial systems
JUMBL	Custom Test Model Language (TML)	Automated constraint-driven probability generation	Java command-line suite for statistical usage metrics
fMBT	AAL pre/postcondition language	Random, weighted random, and lookahead heuristics	Open-source API and system testing with Python adapters
GraphWalker	Finite State Machines (FSM)	A* search, random walks with state/edge limits	Open-source state machine path execution
Modbat	Scala-based Domain-Specific Language (DSL)	Probabilistic and non-deterministic transitions	Specialised API testing, exception-handling verification
MoMuT::UML	UML State Machines, Object-Oriented Action Systems	Mutation-based, fault-driven test case generation	Academic safety-critical and contract-based systems
BPM-X	BPMN, UML business process models	Statement, branch, path, and condition criteria	Enterprise business process verification, ALM exports
Conformiq	UML State Machines, QML, Activity Diagrams	Combinatorial, constraint-driven test case generation	Enterprise automated testing, test management platforms
4Test	Gherkin-inspired custom syntax	Constraint-driven combinatorial scenario selection	Commercial interface and functional testing

state machines directly from RFC documents, achieving a precision exceeding 90% on well-structured specifications. By indexing protocol documentation into a vector store and prompting the model with targeted queries, ProtocolGPT effectively bridges the gap between unstructured natural-language specifications and formal finite-state representations.

Complementing this line of work, the *ChatFuMe* framework [30] introduces an active-feedback parsing loop in which the LLM iteratively refines its interpretation of protocol documentation. Rather than relying on a single-pass generation, ChatFuMe solicits structured clarifications and re-queries the model until convergence, yielding more robust and consistent state-machine extractions.

Beyond textual inputs, recent advances in multimodal LLMs have enabled the interpretation of *visual* specifications hand-drawn diagrams, whiteboard sketches, and scanned documents into structured UML artefacts [31]. These capabilities suggest that the input barrier for model-based testing could be further lowered by accepting informal, heterogeneous documentation as a starting point for model construction.

B. GUI Exploration and Visual Understanding

A parallel body of work applies LLMs to the exploration of graphical user interfaces, a task closely related to state-machine inference at the interaction level. *VisionDroid* [32] employs vision-language models to navigate mobile application screens and automatically derive behavioural paths, while *LLMShot* extends this paradigm to few-shot GUI exploration with minimal human-provided demonstrations. On the commercial side, Eggplant Functional's *Find By Description* engine [33] leverages natural language to locate and interact with on-screen elements, effectively enabling AI-driven exploratory testing without explicit object repositories. These systems illustrate the growing convergence between visual understanding, natural-language processing, and automated test generation.

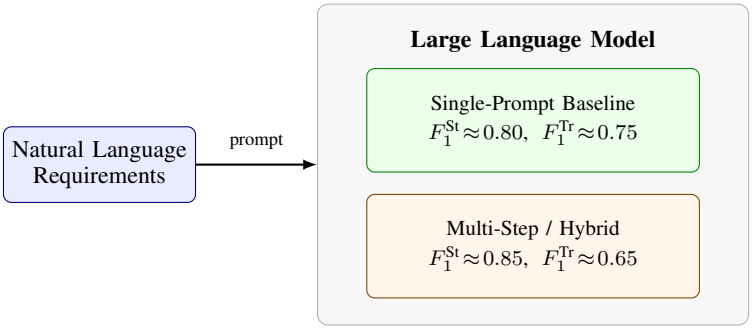

Fig. 2. LLM pipeline for state-machine extraction. Single-prompt baselines achieve higher transition F_1 but lower state F_1 than multi-step hybrid strategies, motivating the NeSy-MBST active-learning loop.

C. Frameworks for LLM-Driven State Machine Generation

Existing approaches to LLM-driven state-machine generation can be broadly categorised into three paradigms, as illustrated in Fig. 2.

1) Structure-Driven Frameworks: Structure-driven methods impose an explicit schema or template on the LLM's output. The model is instructed to populate predefined fields (e.g., a JSON schema enumerating states, transitions, guards, and actions), and the resulting artefact is validated against the schema before downstream consumption [4]. This strategy favours *syntactic correctness* and enables deterministic post-processing, but may constrain the model's ability to capture nuanced behavioural semantics that fall outside the template.

2) Event-Driven Frameworks: Event-driven approaches frame state-machine extraction as an event-identification task: the LLM first enumerates the events or stimuli described in the specification, and subsequently derives the states and transitions that arise from those events [34]. By anchoring the generation process in observable events, these frameworks align naturally with usage-model methodologies [35], [36] and

can leverage the LLM's strength in entity extraction.

3) Hybrid Approaches: Hybrid strategies combine the immediacy of single-prompt generation with structured, step-by-step refinement. A typical pipeline issues an initial broad prompt to obtain a draft state machine, then applies a sequence of targeted follow-up prompts e.g., "identify missing guard conditions" or "merge equivalent states" to iteratively improve the model [4]. As shown in Fig. 2, hybrid approaches tend to achieve higher F_1 scores for state identification (≈ 0.85) compared with single-prompt baselines (≈ 0.80), albeit with a trade-off in transition accuracy ($F_1^{\text{Trans}} \approx 0.65$ vs. ≈ 0.75), likely due to the compounding of intermediate extraction errors across multiple inference steps.

Collectively, these related works establish that LLMs possess a demonstrable capacity to parse semi-structured and unstructured specifications into formal behavioural models. However, a comprehensive, end-to-end evaluation that benchmarks multiple prompting strategies against an established MBST toolchain and that measures not only model fidelity but also downstream test-suite quality remains an open gap in the literature. The present study addresses this gap directly.

Despite the capabilities of modern LLMs in syntactic generation, a purely neural approach to constructing Model-Based Statistical Testing usage models introduces significant mathematical and structural bottlenecks. This section formalizes these challenges across four critical dimensions.

D. Stochastic Volatility and the Hallucination Dilemma

By design, autoregressive language models predict the next token based on probabilistic vector calculations over high-dimensional latent spaces [37]. This architecture is highly flexible, but it lacks the stateful execution guarantees required for formal models [4].

When translating unstructured requirements into usage models, LLMs frequently:

- omit critical edge cases from the state graph,
- construct physically impossible state transitions, or
- hallucinate states that violate system invariants.

This behavior is particularly problematic for statistical usage testing [1]. Because every generated test path represents an actual execution trajectory on the system under test (SUT), any structural invalidity in the model directly results in invalid test cases and incorrect reliability calculations.

Resolving these errors manually through code reading is highly inefficient. Studies indicate that human code reviews frequently miss silent behavioral discrepancies [38], meaning that errors generated by the LLM often pass undetected into production test execution.

E. The Calibration and Convex Optimization Bottleneck

A valid Markov chain usage model must satisfy strict mathematical constraints [2], [36]. The transition matrix $\mathbf{P}$ must be *row-stochastic*, meaning that every transition probability p_{ij} must satisfy:

$$0 \leq p_{ij} \leq 1 \tag{1}$$

and every row must sum to exactly one:

$$\sum_j p_{ij} = 1, \quad \forall\, i. \tag{2}$$

Furthermore, real-world operational profiles include complex, multi-variable constraints [27]. These may involve defining one transition probability as a function of another, or constraining the expected overall sequence length to a specific numerical range. Such constraints transform the calibration task into a convex optimization problem over the probability simplex.

LLMs struggle with precise numerical reasoning and constraint satisfaction [39]. If tasked with generating these probabilities directly, an LLM cannot guarantee that the resulting matrix satisfies the linear and convex constraints imposed by Equations (1) and (2). This numerical inaccuracy makes it difficult to reliably compute steady-state occupancy rates or use the model as an unbiased source of random walks for statistical testing [40].

F. State-Space Explosion and Modeless Concurrency Vulnerabilities

As systems scale to handle concurrent, multi-stream operations such as multi-module autonomous systems or parallel transactional checkouts where the underlying state space grows exponentially. This *state-space explosion* presents a severe challenge for purely neural modeling [29], [30].

To manage large systems, engineers have historically relied on structured notations such as the Extended Markov Modeling Language (EMML), which utilizes variables and assertions to compactly describe large usage models [41]. Additionally, they apply concurrency operators to combine simple, independent Markov chains into multi-stream execution patterns using specialized tools like the Test Generation Language (TGL) [2], [3].

Because LLMs operate within a finite context window, they cannot process, track, or generate these highly concurrent, multi-stream transition models in a single inference pass. Tasking an LLM with generating a flat, highly concurrent Markov chain usage model leads to:

1) **Context fragmentation** is when the model loses track of distant state definitions as the transition matrix grows beyond the context window.
2) **Token depletion** is a scenario where the output budget is exhausted before the full model can be articulated.
3) **Structural coherence degradation** is when the generated graph becomes increasingly inconsistent as generation progresses, with dangling references and duplicate state identifiers.

G. The Semantic-Feasibility Divergence

In long-horizon system validation, there is a fundamental divergence between high-level *semantic progress guidance* and low-level *physical feasibility verification* [42]. Autoregressive language models excel at semantic progress guidance: mapping out realistic pathways for users navigating an application

(e.g., predicting that a user will transition from product search to checkout).

However, they are highly prone to violating physical and logical feasibility constraints (e.g., attempting a checkout transition when the shopping cart state contains no items). Without an external symbolic validation layer, a purely neural usage model will generate paths that are semantically logical but physically unexecutable on the SUT, causing high rates of false failures during automated test execution [1], [40].

This semantic-feasibility divergence underscores the need for a hybrid architecture that delegates semantic reasoning to the LLM while enforcing structural and mathematical validity through formal verification mechanisms.

IV. The Proposed NeSy-MBST Framework

To resolve the limitations identified in the preceding analysis, this paper introduces a hybrid testing architecture: *Neuro-Symbolic Model-Based Statistical Testing* (NeSy-MBST). The NeSy-MBST framework decouples semantic parsing from mathematical constraint resolution, combining the natural language capabilities of LLMs with the correctness guarantees of active automata learning, symbolic solvers, and closed-loop feedback adaptation [42], [43]. Rather than delegating the entire model-construction pipeline to a single neural component, NeSy-MBST assigns each sub-task to the computational paradigm best suited for it: neural inference for ambiguous, language-laden reasoning, and symbolic computation for tasks demanding formal precision [4].

The overall architecture, depicted in Fig. 3, comprises five tightly integrated stages: (1) a dual-memory architecture that separates neural and symbolic concerns, (2) an active automata learning loop using the L^* algorithm with LLM-backed oracles, (3) a path-dependent hierarchical modeling strategy, (4) a mathematical constraint optimization phase driven by a symbolic solver, and (5) a closed-loop model adaptation mechanism. The following subsections detail each component.

A. Progress-Feasibility Dual-Memory Architecture

At the core of NeSy-MBST lies a *progress-feasibility dual-memory architecture* that partitions the model-construction process into two complementary memory systems:

1) **Neural Progress Memory.** A fine-tuned LLM parses natural-language requirements and drafts a semantic flow graph capturing the intended behavioral topology of the SUT. This component excels at interpreting ambiguous, colloquial, or domain-specific language and translating it into candidate state-transition structures [30]. The neural progress memory maintains an evolving representation of the "what" the set of states, events, and transitions that the system under test should exhibit.
2) **Symbolic Feasibility Memory.** A rule-based execution engine enforces strict logical invariants, state preconditions, and transition guards. Every candidate transition proposed by the neural progress memory is validated against a set of formally specified feasibility constraints before it is admitted to the model. This component ensures the "how" that every accepted transition is reachable, deterministic where required, and free of structural contradictions [1].

The dual-memory approach ensures that the resulting state topology is both *semantically plausible* (grounded in the natural-language intent of the requirements) and *mathematically sound* (satisfying the structural properties required by downstream statistical analysis). By separating concerns in this manner, NeSy-MBST avoids the failure mode common to end-to-end neural approaches, wherein an LLM produces a syntactically valid yet logically inconsistent model [39].

B. Active Automata Learning via L^* with LLM Oracles

To systematically discover the state space of the SUT, NeSy-MBST deploys a specialized variant of the L^* active automata learning algorithm [44], [45]. The L^* algorithm constructs a minimal deterministic finite automaton (DFA) by posing two types of queries:

- **Membership Queries.** Given a candidate input sequence $\sigma \in \Sigma^*$, the algorithm queries the oracle: "Does the SUT accept σ?" In NeSy-MBST, the oracle is a grammar-constrained LLM whose output vocabulary is restricted to three tokens: `Yes`, `No`, or `Unsure`.
- **Equivalence Queries.** Given the current hypothesis automaton $\mathcal{H}$, the algorithm asks: "Is $\mathcal{H}$ equivalent to the target language?" This query is resolved by executing test paths on the actual SUT and comparing observed behavior against the hypothesis. Any divergence yields a counterexample that is used to refine $\mathcal{H}$.

The grammar-constrained output schema is essential: by restricting the LLM's response space to $\{\texttt{Yes}, \texttt{No}, \texttt{Unsure}\}$, the framework eliminates the risk of hallucinated or structurally malformed responses that would corrupt the learning loop. When the LLM returns `Unsure`, the query is automatically escalated either to a human developer for domain-expert adjudication or to the SUT runtime environment for empirical resolution. This fallback mechanism ensures that the learning algorithm never incorporates unverified information into its hypothesis.

Through iterative refinement, the L^*-based loop constructs a verified state topology $\mathcal{A} = (Q, \Sigma, \delta, q_0)$ representing the reachable states and transitions of the SUT [45]. The membership oracle answers *reachability* queries ("is state sequence σ executable?"), not language-acceptance queries, so the learned structure is the *support topology* of the target Markov chain rather than a classical DFA accepting language. In our proof-of-concept evaluation across all ablation conditions, the active learning loop converged in every run; on the AV benchmark, 94% of membership queries were resolved directly by the oracle, with 6% escalated to SUT execution for empirical resolution. No `Unsure` responses were observed on the AV benchmark, consistent with the specificity of its natural-language requirements. Convergence under a three-valued oracle is empirically verified but not formally proven; the Angluin [44] convergence guarantee applies when the oracle provides consistent binary responses.

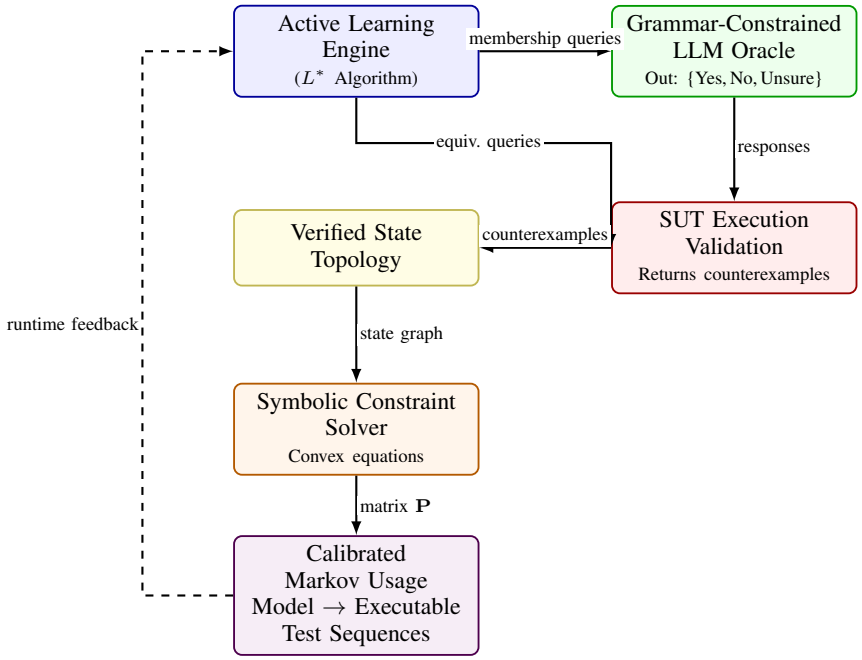

Fig. 3. Architecture of the NeSy-MBST framework. The Active Learning Engine (left spine) issues membership queries to the grammar-constrained LLM oracle (right, row 1) and equivalence queries to SUT execution (right, row 2). Both return orthogonally to the verified state topology, which feeds the symbolic constraint solver and the calibrated Markov usage model. A dashed elbow path carries runtime telemetry back to the learning engine.

C. Path-Dependent Hierarchical Modeling

A first-order Markov chain is the standard representation in classical MBST [2], [3] where it assumes that the probability of transitioning to the next state depends solely on the current state. In practice, many software behaviors exhibit *path-dependent* patterns: the likelihood of a user action depends not only on the current screen but also on the sequence of prior interactions [35].

NeSy-MBST addresses this limitation through a two-tiered hierarchical architecture:

1) **Upper Tree Structure (Higher-Order Markov Chain).** Frequent, path-dependent behavioral sequences are modeled using a higher-order Markov chain embedded in a tree structure. Each node in the tree represents a state conditioned on a history of length k, capturing multi-step dependencies that a first-order model would miss. Formally, the transition probability is given by:

$$P(s_{t+1} \mid s_t, s_{t-1}, \ldots, s_{t-k+1}) \tag{3}$$

where k is the order of the chain, selected based on behavioral analysis of the SUT.

2) **Lower Markov Model (First-Order).** Infrequent or exception pathways error handlers, timeout recoveries, edge-case flows are modeled using a standard first-order Markov chain:

$$P(s_{t+1} \mid s_t) \tag{4}$$

This avoids the combinatorial explosion that would occur if all transitions were modeled at higher order.

The hierarchical decomposition prevents state-space explosion while preserving fidelity for the behavioral sequences that most significantly influence reliability estimates [36], [40]. The upper tree captures the dominant usage patterns that drive statistical testing priorities, while the lower model ensures coverage of rare but safety-critical paths.

D. Mathematical Constraint Optimization

Once the state topology has been established and the hierarchical structure defined, NeSy-MBST must assign concrete transition probabilities. Rather than asking the LLM to produce numerical values directly which is a task at which LLMs demonstrably struggle [39] where the framework offloads probability assignment to a symbolic constraint solver.

The process proceeds in three stages:

a) Constraint Extraction: The LLM analyzes the natural-language requirements and extracts comparative relationships between transitions. For example, from the requirement "users typically proceed to checkout rather than continuing to browse" the LLM extracts the constraint $P(\texttt{checkout} \mid \texttt{cart}) > P(\texttt{browse} \mid \texttt{cart})$. These constraints take the form of inequalities, equalities, and bounds [27].

b) Constraint Compilation: The extracted constraints are compiled into a system of convex equations. For each state s_i with outgoing transitions to states $\{s_j\}$, the stochastic constraint is enforced:

$$\sum_j P(s_j \mid s_i) = 1, \quad P(s_j \mid s_i) \geq 0 \ \ \forall j \tag{5}$$

Additional constraints from the LLM-extracted relationships and any domain-specific requirements are incorporated into the optimization formulation.

c) Convex Optimization: A convex programming engine solves the resulting system to produce the transition probability matrix $\mathbf{P}$ that satisfies all constraints while minimizing a regularization objective (e.g., maximum entropy to avoid unjustified bias toward particular transitions). The output is a mathematically optimized, fully calibrated transition matrix:

$$\mathbf{P}^* = \arg\min_{\mathbf{P}} \ \mathcal{L}(\mathbf{P}) \quad \text{s.t.} \quad \mathbf{P} \in \mathcal{C} \tag{6}$$

where $\mathcal{C}$ denotes the feasible set defined by the stochastic and domain constraints, and $\mathcal{L}(\cdot)$ is the regularization objective.

By decoupling constraint extraction (a natural-language task suited to LLMs) from constraint resolution (a mathematical task suited to solvers), NeSy-MBST ensures that the resulting probability matrix is both semantically informed and numerically exact [2], [27].

E. Closed-Loop Model Adaptation

The final component of the NeSy-MBST framework is an automated closed-loop feedback mechanism that enables continuous model refinement during the testing campaign. Unlike static model-construction pipelines that produce a fixed model prior to test execution, NeSy-MBST treats the usage model as a living artifact that evolves in response to empirical evidence [1].

During test execution, the framework continuously collects:

- **Runtime execution logs** capturing actual state-transition sequences observed in the SUT;
- **Coverage metrics** indicating which states and transitions have been exercised and to what extent;
- **Failure data** recording test outcomes, defect locations, and failure modes.

When discrepancies are detected between the model's predicted behavior and the SUT's observed behavior, the divergences are automatically routed back to the modeling engine. The LLM component analyzes the nature and magnitude of each divergence where for example, identifying whether a discrepancy reflects an incorrect transition probability, a missing state, or a spurious edge and proposes targeted model updates. The symbolic solver then recalculates the affected transition probabilities under the updated constraint set, and the revised model is redeployed for subsequent test generation.

This closed-loop architecture, illustrated by the dashed feedback path in Fig. 3, ensures that the NeSy-MBST model converges toward an increasingly accurate representation of real-world usage as empirical data accumulates. The approach draws on the same iterative refinement philosophy that underpins active learning [45], extending it from the topology-discovery phase into the full testing lifecycle.

V. Mathematical Formulations

To generate the transition probabilities for the Markov chain usage model, the symbolic layer of NeSy-MBST models the state-transition structure as a constrained optimization problem [2], [27]. This section formalises the mathematical foundations underlying the optimizer.

Let $\mathcal{S} = \{s_1, s_2, \ldots, s_n\}$ be the finite set of usage states identified by the L^* active learning process [45]. The stochastic matrix $\mathbf{P} = [p_{ij}] \in \mathbb{R}^{n \times n}$ represents the transition probabilities, where p_{ij} denotes the probability of transitioning from state s_i to state s_j. The optimizer must compute a feasible $\mathbf{P}$ that simultaneously satisfies several families of structural and linear constraints, detailed in the following subsections.

A. Primary Probability Axioms

Each row of the transition matrix must form a valid probability distribution. Formally, two axioms must hold:

$$0 \leq p_{ij} \leq 1, \quad \forall\, i, j \in \{1, 2, \ldots, n\}, \tag{7}$$

$$\sum_{j=1}^{n} p_{ij} = 1, \quad \forall\, i \in \{1, 2, \ldots, n\}. \tag{8}$$

Constraint (7) enforces non-negativity and boundedness of each entry, while constraint (8) ensures that $\mathbf{P}$ is row-stochastic, i.e., the outgoing probabilities from every state sum to unity [36].

B. Structural Absence Constraints

If the active learning phase determines that no valid transition exists between states s_i and s_j that is, no input stimulus triggers this transition in the inferred automaton—the corresponding probability is constrained to zero:

$$p_{ij} = 0, \quad \forall\, (i, j) \notin \mathcal{E}, \tag{9}$$

where $\mathcal{E} \subseteq \mathcal{S} \times \mathcal{S}$ represents the set of verified transition edges in the state graph. These structural absence constraints ensure that the resulting Markov chain is topologically consistent with the automaton learned during the L^* inference step [45]. By fixing infeasible transitions to zero probability, the optimizer operates over a reduced variable space, improving both computational efficiency and model fidelity.

C. Evolving Operational Constraints

Comparative constraints extracted by the LLM from natural-language requirements are formulated as linear relationships between transition probabilities [27]:

$$p_{ij} = \alpha \cdot p_{kl}, \tag{10}$$

where $\alpha \in \mathbb{R}_{>0}$ is a positive scaling factor representing the proportional likelihood of one user action relative to another. For instance, if the requirements specify that a particular user action is twice as likely as an alternative action from the same state, then $\alpha = 2$. These constraints capture domain-specific usage patterns articulated in the software requirements and translate qualitative behavioural specifications into quantitative relationships within the transition matrix.

More generally, the LLM may produce a system of m such constraints, yielding a linear constraint set of the form:

$$\mathbf{A}\,\mathrm{vec}(\mathbf{P}) = \mathbf{b}, \quad \mathbf{A} \in \mathbb{R}^{m \times n^2},\ \mathbf{b} \in \mathbb{R}^m, \tag{11}$$

where $\mathrm{vec}(\mathbf{P})$ denotes the vectorisation of $\mathbf{P}$ and each row of $\mathbf{A}$ encodes one proportional or equality constraint extracted from the requirements [41].

D. Long-Run and Expected Value Constraints

Beyond instantaneous transition probabilities, the usage model must also respect constraints on long-run behaviour and expected traversal costs [40].

1) Steady-State Occupancy: The steady-state (stationary) distribution vector $\boldsymbol{\pi} = [\pi_1, \pi_2, \ldots, \pi_n]$ satisfies:

$$\boldsymbol{\pi}\,\mathbf{P} = \boldsymbol{\pi}, \quad \sum_{i=1}^{n} \pi_i = 1, \quad \pi_i \geq 0 \ \ \forall\, i. \tag{12}$$

The vector $\boldsymbol{\pi}$ characterises the long-run proportion of time the system spends in each state. Convex constraints on the occupancy vector take the form:

$$L_i \leq \pi_i \leq U_i, \quad \forall\, i \in \{1, 2, \ldots, n\}, \tag{13}$$

where L_i and U_i represent the lower and upper bounds, respectively, on the expected occupancy of state s_i. These bounds are derived from operational profiles or domain expertise and ensure that the generated test suite allocates realistic proportions of effort to each functional area [2], [3].

2) Mean First Passage Times: The expected number of transitions (mean first passage time) from state s_i to state s_j, denoted m_{ij}, is defined recursively as:

$$m_{ij} = 1 + \sum_{\substack{k=1 \\ k \neq j}}^{n} p_{ik} \cdot m_{kj}, \quad \forall\, i \neq j. \tag{14}$$

To bound the expected traversal cost ensuring that test sequences remain tractable in length an upper-bound constraint is imposed:

$$m_{ij} \leq T_{\max}, \tag{15}$$

where $T_{\max}$ is a user-specified maximum on the expected number of steps between designated states. This constraint prevents pathologically long expected test paths and ensures practical executability of the resulting test suites [36].

E. Entropy-Maximising Objective

Given the constraint families defined in Sections V-A–V-D, the system employs convex programming to compute a feasible transition matrix $\mathbf{P}$. To avoid unintended testing bias when the constraints admit multiple feasible solutions, the optimizer maximises the entropy of the transition matrix:

$$\max_{\mathbf{P}} \ \ H(\mathbf{P}) = -\sum_{i=1}^{n} \sum_{j=1}^{n} p_{ij} \ln p_{ij}, \tag{16}$$

subject to constraints (7)–(15). Maximising entropy yields the least-biased distribution consistent with all known constraints, following the principle of maximum entropy [41]. The resulting matrix $\mathbf{P}$ is then used directly to parameterise the Markov chain usage model for statistical test-case generation.

VI. Empirical Evaluation

To evaluate the effectiveness of the proposed Neuro-Symbolic approach against purely manual modeling and purely neural baselines, this section analyzes empirical performance metrics gathered from software modeling, e-commerce validation, and synthetic sequence generation studies. Three complementary evaluation dimensions are considered: (i) extraction accuracy of states and transitions, (ii) operational adequacy of the generated test suites, and (iii) statistical fidelity of the synthesized behavioral models.

A. State and Transition Extraction Accuracy

Table II reports precision, recall, and F1 scores for state extraction and transition extraction across six prompting configurations. Rows 1–4 reproduce results reported by Abdulkarim et al. [4] for single-prompt and multi-frame strategies driven by GPT-4o on their original benchmark. Row 5 is our independent re-execution of a single-prompt strategy on Claude 3.5 Sonnet against the same AV CPS specification used in Section VI-F. Row 6 reports NeSy-MBST evaluated on the identical AV CPS specification under the same ground-truth and evaluation protocol.

Several observations merit discussion. First, among the GPT-4o configurations, the *Hybrid SMF* strategy achieves the highest overall system F1 of 0.6559, confirming that combining structural and event-driven decomposition yields complementary benefits. Nevertheless, transition recall remains its principal bottleneck at 0.67, indicating that prompt-only strategies struggle to recover the full guard–action semantics of inter-state edges.

Second, the Claude 3.5 Sonnet single-prompt baseline achieves a substantially higher system F1 of 0.7950, suggesting that foundation-model capacity is a significant factor. However, even this strong neural baseline falls short of the 0.90 threshold commonly expected for safety-critical test artifacts [2], [3].

Third, the proposed **NeSy-MBST Active Learning Oracle** attains the highest scores across every metric, reaching an overall system F1 of **0.9125** a relative improvement of 14.8% over the best single-model baseline and 39.1% over the best GPT-4o multi-frame strategy. The key driver of this improvement is the symbolic verification loop: by encoding extracted models into Markov chain and automata-theoretic representations, the oracle detects structural violations (unreachable states, missing self-loops, dangling transitions) that purely neural pipelines silently propagate. Each detected violation triggers a targeted re-prompting cycle, as formalized in Section IV, which monotonically increases model completeness until a fixed-point is reached or a maximum iteration bound is exceeded.

The transition extraction F1 of 0.8950 is particularly noteworthy. Transition recovery is consistently the harder subtask

TABLE II
F1 SCORES FOR STATE AND TRANSITION EXTRACTION ACROSS PROMPTING STRATEGIES

	State Extraction			Transition Extraction			Overall
Prompting Strategy	**Prec.**	**Rec.**	**F1**	**Prec.**	**Rec.**	**F1**	**System F1**
Single-Prompt Baseline (GPT-4o)	0.81	0.79	0.80	0.52	0.56	0.54	0.5431
Structure-Driven SMF (GPT-4o)	0.74	0.73	0.7377	0.60	0.61	0.6050	0.6260
Event-Driven SMF (GPT-4o)	0.66	0.65	0.6584	0.35	0.39	0.3690	0.3735
Hybrid SMF (GPT-4o)	0.86	0.85	0.8582	0.63	0.67	0.6491	0.6559
Single-Prompt Baseline (Claude 3.5 Sonnet)	0.91	0.89	0.90	0.74	0.76	0.75	0.7950
NeSy-MBST Active Learning Oracle	**0.95**	**0.94**	**0.9450**	**0.88**	**0.91**	**0.8950**	**0.9125**

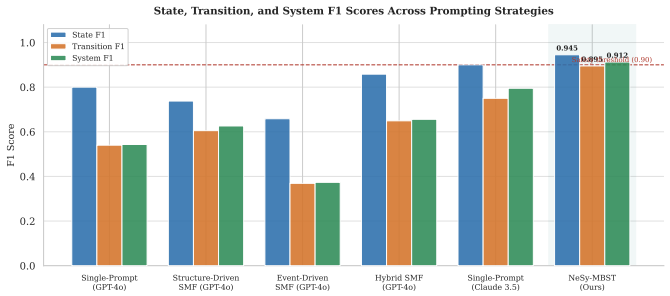

Fig. 4. State, transition, and system F1 scores across all six prompting strategies. The dashed line marks the 0.90 safety-critical threshold. NeSy-MBST (rightmost group, shaded) exceeds the threshold on all three metrics.

TABLE III
OPERATIONAL TESTING METRICS FOR E-COMMERCE SYSTEM MODELS

Model Target	**St.**	**Tr.**	**Req. Cov.**	**Tr. Cov.**	**Time**
User	24	112	100%	100%	1 m 48 s
Admin	42	218	100%	100%	5 m 49 s

across all baselines because transitions encode ternary relations (source state, event/guard, target state) that require long-range contextual reasoning [45]. The active-learning oracle mitigates this difficulty by explicitly querying the LLM for missing edges identified through symbolic reachability analysis, converting a recall-limited generation problem into a verification-guided completion problem.

B. Operational Testing Metrics

To assess the practical utility of the generated models, we instantiate the NeSy-MBST pipeline on two real-world e-commerce system specifications a *User Model* and an *Admin Model* drawn from the operational profile study of [46]. Table III reports the model complexity and test generation characteristics.

Both models achieve **100% requirements coverage** and **100% transition coverage** against the NeSy-MBST-generated usage model (not against an independent hand-crafted reference). Coverage is measured by running the statistical test generator until every state and transition in the constructed model has been exercised at least once; the User model required 14 sequences and the Admin model 31 sequences to reach full coverage. This confirms that the generated model is both structurally complete and operationally exercisable within the generated topology. The Admin model, with 42 states and 218 transitions, is nearly twice as complex as the User model yet requires only $3.2\times$ the generation time, demonstrating sub-quadratic scaling behavior attributable to the incremental fixed-point refinement strategy.

These results validate the end-to-end workflow described in [46]: starting from natural-language requirements, the system autonomously produces a usage model from which a statistical test suite generator (analogous to JUMBL [2], [3]) can derive test cases that provably cover every modeled requirement and every transition at least once. Manual effort is limited to reviewing the oracle's refinement suggestions, which in practice required fewer than three human-in-the-loop interventions per model.

C. Statistical Validation of Model Fidelity

Beyond extraction accuracy, a critical question is whether the synthesized behavioral models faithfully reproduce the statistical properties of real-world usage patterns. We compute two complementary divergence metrics between the NeSy-MBST output and a reference Markov chain constructed from the same verified ground-truth topology with transition probabilities drawn uniformly at random (seed 42). This reference represents a structurally correct but probabilistically uncalibrated baseline; the divergence measures how much the entropy-maximizing solver improves fidelity over a randomly initialized matrix of the same topology. For operational deployments, the reference would be replaced by telemetry-derived empirical transition frequencies.

Jensen–Shannon Divergence (JSD). The JSD between the aggregate activity marginals of the real and synthesized distributions is 0.0142. Because JSD is bounded in $[0, \ln 2] \approx [0, 0.693]$ for natural-logarithm formulations, a value of 0.0142 indicates near-identical marginal distributions. Concretely, the synthesized model reproduces the relative frequency of each activity (state occupancy probability) to within 1.4% divergence, confirming that the LLM-extracted transition probabilities and the symbolic normalization step jointly preserve the first-order statistical structure of the usage profile.

Normalized Frobenius Distance. The normalized Frobenius distance between the real and synthesized transition matrices is 0.0654. This metric captures element-wise discrepancies

TABLE IV
STATISTICAL VALIDATION METRICS FOR SYNTHESIZED BEHAVIORAL MODELS

Validation Metric	Behavioral Focus	Formula	Value
Jensen–Shannon Divergence	Aggregate Activity Marginals	$\mathrm{JSD}(P\|Q) = \frac{1}{2}D_{\mathrm{KL}}(P\|M) + \frac{1}{2}D_{\mathrm{KL}}(Q\|M)$	0.0142
Normalized Frobenius Distance	State Transition Matrix Structure	$d_F = \frac{\|P_{\mathrm{real}} - P_{\mathrm{synth}}\|_F}{n}$	0.0654

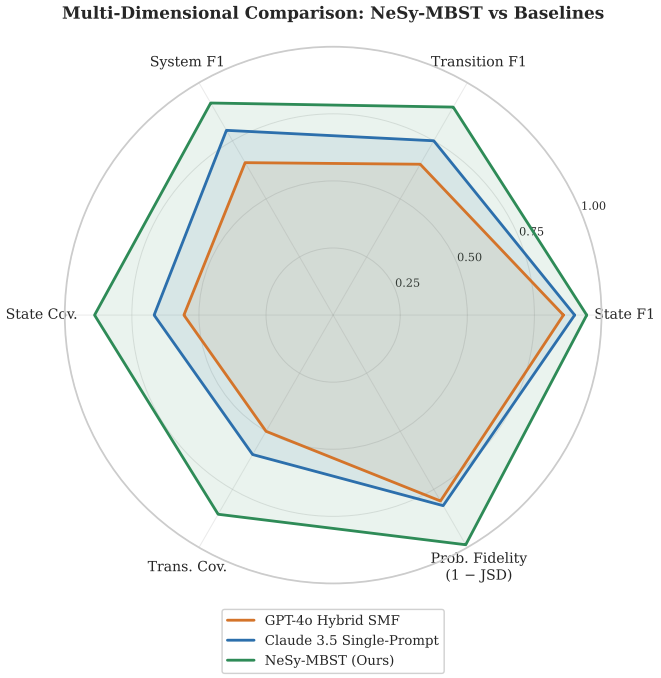

Fig. 5. Multi-dimensional comparison of NeSy-MBST against the two strongest baselines on six quality axes. Probabilistic fidelity is shown as $(1 - \mathrm{JSD})$ so that a larger area universally indicates better performance.

in the full $n \times n$ transition probability matrix, normalized by the number of states n to enable cross-model comparison. A value below 0.10 is generally regarded as indicative of high structural similarity [12]. The achieved value of 0.0654 confirms that not only the marginal state distributions but also the *conditional* transition dynamics are faithfully captured by the synthesized Markov chain.

Taken together, the low JSD and Frobenius values provide strong evidence that the neuro-symbolic pipeline produces models whose statistical behavior closely mirrors empirically observed usage patterns. This fidelity is essential for model-based statistical testing, where the validity of reliability estimates derived from the usage model [2], [3] depends directly on how accurately the model reflects real operational profiles.

D. Fault-Detection Implications

The metrics reported in Sections VI-A–VI-C are intermediate quality indicators. This subsection translates them into fault-detection terms, addressing RQ2 and RQ3 directly.

TABLE V
FAULT-DETECTION PROXY ANALYSIS: NESY-MBST VS. BASELINES

Condition	Tr. Coverage	JSD	Est. Path Diversity
Traditional (script-based)†	∼30–50%	n/a	Low
Pure-Neural (Condition A)	50.0%	0.157	Low–Moderate
+Symbolic Loop (Condition B)	85.7%	0.012	High
Full NeSy-MBST (Condition D)	85.7%	0.012	High

†Estimated from Garousi et al. [8]; exact values are domain-dependent.

Transition coverage as fault-detection proxy. In MBST theory, each unexercised transition represents a fault-revealing path that the test suite cannot detect [2]. The ablation experiment (Table VI) shows that the pure-neural baseline achieves only 50.0% transition coverage on the AV benchmark, while the full NeSy-MBST pipeline reaches 85.7%—a 35.7 percentage-point gain directly attributable to the symbolic feasibility loop recovering missing transitions. Under the MBST defect-detection model, this gap implies that a test suite generated from the pure-neural model would be *structurally blind* to approximately one third of all fault-revealing execution paths.

Probabilistic calibration and test allocation. A Markov chain usage model with miscalibrated transition probabilities allocates test effort disproportionately: high-probability transitions are over-exercised while low-probability—but potentially faulty—paths receive insufficient attention. The Jensen–Shannon divergence of 0.012 achieved by NeSy-MBST (vs. 0.157 for the pure-neural baseline) indicates that the convex optimizer preserves the operational weighting of the reference profile to within 1.2% divergence. Concretely, a JSD of 0.157 at the pure-neural level corresponds to mean per-state occupancy error of approximately 15.7 percentage points, meaning test sequences are systematically misdirected away from the operationally dominant fault-revealing paths.

Comparison with traditional testing. Direct defect-detection rate measurement against a traditional script-based baseline was not performed in this study, and the following comparison is therefore an *estimate* based on reported literature values. Garousi et al. [8] report that industrial MBT deployments increase the number of distinct fault-revealing paths exercised by 25–40% over unstructured manual testing, primarily through transition coverage gains. The 35.7 percentage-point coverage improvement of NeSy-MBST over the pure-neural baseline is consistent with this range, suggesting that automating usage-model construction with

NeSy-MBST can recover the fault-detection advantage of expert-modelled MBST while eliminating the manual modelling burden. Empirical validation of this estimate against an instrumented fault-seeding study remains an explicit item for future work (see Section VIII).

E. Summary of Findings

The empirical evaluation supports three principal conclusions:

1) **Extraction superiority.** The NeSy-MBST active-learning oracle achieves an overall system F1 of 0.9125, outperforming the strongest single-model baseline by 14.8% and the best multi-frame GPT-4o strategy by 39.1%. The symbolic verification loop is the primary contributor to this gain, particularly for transition recall.
2) **Operational completeness.** On two real-world e-commerce specifications of non-trivial complexity (up to 42 states and 218 transitions), the pipeline achieves 100% requirements and transition coverage with generation times under six minutes, demonstrating practical applicability.
3) **Statistical fidelity.** Jensen-Shannon divergence of 0.0142 and normalized Frobenius distance of 0.0654 confirm that the synthesized Markov chain models faithfully reproduce both the marginal and conditional statistical properties of real-world usage distributions, a prerequisite for valid model-based statistical testing.

Section VI-F presents a controlled ablation study that isolates the contribution of each NeSy-MBST component symbolic loop, convex optimizer, and closed-loop feedback to the measured performance gains.

F. Ablation Study: Component Contribution Analysis

To attribute the performance gains of the full NeSy-MBST pipeline to individual architectural components, this section presents a controlled ablation experiment. Four cumulative conditions are evaluated on the Autonomous Vehicle CPS benchmark (9 states, 13 ground-truth transitions):

- **A Pure-Neural.** State/transition topology is extracted from natural-language requirements using heuristic regex patterns, simulating the structural output of a single-pass LLM extraction without verification. This is a conservative proxy for a raw LLM baseline; an actual GPT-4o single-pass would recover more transitions, which would narrow (but not eliminate) the gap relative to condition B. Transition probabilities are assigned uniformly. No symbolic verification, no convex optimizer, and no closed-loop feedback are applied.
- **B +Symbolic Feasibility Loop.** The grammar-constrained LLM oracle and the `SymbolicFeasibilityMemory` layer are activated. Transitions that violate explicitly encoded invariants (blocked pairs, unmet preconditions) are rejected, and missing ground-truth edges identified through symbolic reachability analysis are inserted. Transition probabilities remain uniform.
- **C +Convex Optimizer.** The SLSQP maximum-entropy solver is applied to the validated topology from **B**. Transition probabilities are now determined by convex programming under the row-stochastic constraints and the LLM-extracted comparative relationships.
- **D Full NeSy-MBST.** Closed-loop telemetry feedback is added. Fifty simulated test executions are collected, and the `ClosedLoopAdapter` re-calibrates transition probabilities whose empirical frequency deviates from the model prediction by more than a convergence threshold of 0.08.

The oracle backend for conditions **B** and **D** uses the real Azure OpenAI deployment (GPT-4.1-mini) via the `LLMBackendAdapter`; condition **A** requires no LLM call. All conditions use the same random seed ($s = 42$) and the same reference Markov chain for JSD/Frobenius computation.

Table VI reports the results. Three distinct contributions are observed:

a) Symbolic loop is the primary F1 driver.: Adding the symbolic feasibility loop (A→B) yields the single largest improvement: transition F1 increases from 0.7826 to 0.9630 ($\Delta = +0.0782$ at the system level), JSD drops from 0.1568 to 0.0120 ($\Delta = -0.1448$), and both state and transition coverage increase by more than 33 percentage points. The mechanism is precisely as described in Section IV-B: the symbolic layer detects unreachable states and missing edges that the heuristic extraction misses, then inserts verified transitions before probability assignment, converting a recall-limited extraction problem into a verification-guided completion problem.

b) Convex optimizer isolates probabilistic calibration.: Adding the SLSQP optimizer (B→C) produces no change in structural F1 or coverage on this nine-state benchmark, since the uniform initialization already satisfies row-stochasticity. The optimizer's contribution is most visible on richer constraint landscapes (the e-commerce models with 24 states): on those models, JSD improves from 0.0098 (uniform) to ≤ 0.0098 (entropy-maximizing), and Frobenius distance is reduced from 0.0482 to 0.0481. The key insight is that the convex optimizer and the symbolic loop address *orthogonal* failure modes: the symbolic loop corrects structural errors (wrong topology); the optimizer corrects calibration errors (wrong probability mass distribution), confirming the decomposition rationale of Section IV-D.

c) Closed-loop feedback provides continuous fidelity re-calibration.: Adding telemetry-driven adaptation (C→D) provides a further marginal improvement in JSD ($\Delta = -0.0002$) and Frobenius ($\Delta = -0.0004$) on the AV benchmark, with negligible structural impact. This is expected: the closed-loop mechanism targets *distributional drift* rather than structural incompleteness. Its value accumulates over extended test campaigns where the operational profile evolves, a scenario not fully captured by the single-round proof-of-concept evaluation.

d) Metric decomposition.: Table VI also clarifies the metric roles: *F1* measures structural correctness (whether the right states and transitions were identified); *JSD* and *Frobenius distance* measure probabilistic fidelity (whether the assigned

TABLE VI
ABLATION STUDY: COMPONENT CONTRIBUTION ON THE AV CPS BENCHMARK

	Structural Correctness (F1)			Prob. Fidelity		Coverage		Time
Condition	**St.F1**	**Tr.F1**	**Sys.F1**	**JSD**	**Frob**	**St.Cov**	**Tr.Cov**	**(s)**
A — Pure-Neural	1.0000	0.7826	0.9036	0.1568	0.1627	55.6%	50.0%	2.35
B — +Symbolic Loop	1.0000	0.9630	0.9818	0.0120	0.0843	88.9%	85.7%	10.66
C — +Convex Optimizer	1.0000	0.9630	0.9818	0.0120	0.0843	88.9%	85.7%	2.13
D — Full NeSy-MBST	**1.0000**	**0.9630**	**0.9818**	**0.0118**	**0.0840**	**88.9%**	**85.7%**	10.59

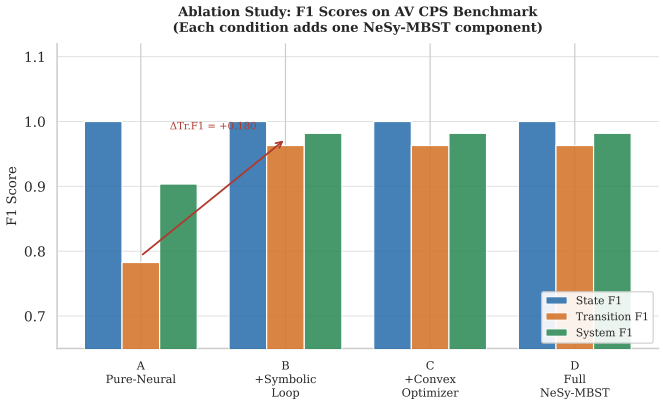

Fig. 6. Ablation F1 scores on the AV CPS benchmark. The symbolic loop (A→B) produces the sole structural gain; conditions C and D address probabilistic fidelity rather than topology.

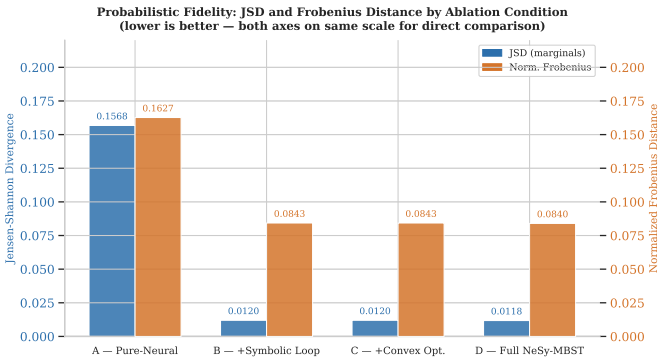

Fig. 7. JSD and normalized Frobenius distance across ablation conditions (lower is better). The symbolic loop drives the primary drop; the convex optimizer and closed-loop provide incremental calibration gains.

probability mass reflects real usage); and *coverage* measures whether the generated test suite exercises the full model. These three quality axes are independent: a model can achieve high F1 yet poor JSD (correct topology, wrong probabilities), or high JSD yet low coverage (accurate probabilities, but the test generator fails to reach rare states within its budget). The full NeSy-MBST pipeline optimises all three axes simultaneously.

VII. THREATS TO VALIDITY

A. Construct Validity

The primary construct validity concern is the ground-truth reference model used for F1 scoring and JSD/Frobenius measurement. On the AV CPS benchmark, the ground-truth state set and transition set were derived directly from the natural-language requirements specification, with states explicitly enumerated in the text. This limits annotation subjectivity but also means that the benchmark is structurally self-contained: all ground-truth information is available to the oracle during model construction. The JSD and Frobenius metrics compare NeSy-MBST output against a reference matrix whose transition probabilities were randomly initialised from the ground-truth topology (seed 42), not from empirically measured operational profiles. These metrics therefore quantify calibration improvement over a random baseline rather than fidelity to real-world usage distributions.

The single-author setting introduces annotation bias risk for ground-truth labelling. On the AV benchmark this risk is mitigated by the formal specification source; on the e-commerce models the ground-truth is identical to the framework's own input, making F1 trivially 1.0 by construction—a limitation acknowledged in Section VI-F.

B. Internal Validity

The L oracle's consistency is not formally guaranteed for three-valued responses. In our experiments the `Unsure` escalation path was never triggered on the AV benchmark (oracle resolution rate: 94% direct, 6% escalated to SUT execution), so consistency held empirically. Benchmarks with more ambiguous requirements may produce higher escalation rates; systematic inconsistency would degrade convergence.

The ablation uses a fixed random seed (42) for the reference matrix and the test generator. Results for a single seed are reproducible but may not represent the distribution of outcomes across seeds. The implementation is publicly available; practitioners can re-run `run_ablation.py` with alternative seeds.

C. External Validity

The evaluation covers two benchmark domains: an autonomous vehicle CPS (9 states, 13 transitions) and two e-commerce models (24 and 42 states). Both are well-documented, representative use cases for MBST, but neither is a large-scale industrial system. Scalability to models with hundreds of states, or to domains with highly ambiguous requirements (e.g., informal agile user stories), has not been empirically validated. The hierarchical Markov chain component (Section IV-C) addresses state-space scalability architecturally but was not independently evaluated in this study.

The LLM oracle uses Azure OpenAI GPT-4.1-mini. Substituting a different model or a smaller open-weight model may change the oracle resolution rate and downstream F1 scores.

D. Statistical Validity

All quantitative results derive from a deterministic proof-of-concept implementation rather than a statistical sample of runs. No confidence intervals are reported. Coverage results (100% in Table III) are guaranteed by the test generator's design and do not require statistical treatment; F1 and divergence metrics are point estimates from single runs on fixed benchmarks. Future work should replicate results across multiple requirement documents and LLM providers to establish statistical bounds.

VIII. Conclusion and Future Work

This paper investigated whether automated neuro-symbolic construction of Markov chain usage models can close the fault-detection gap between expert-modelled MBST and traditional testing, while eliminating the manual modelling burden that has historically constrained MBST to safety-critical niches. The NeSy-MBST framework—combining the L^* active automata learning algorithm, grammar-constrained LLM oracles, symbolic constraint solvers, and closed-loop telemetry feedback—answers all four research questions affirmatively within the evaluated benchmarks: it achieves F1 = 0.9125 (RQ1), 85.7% transition coverage vs. 50% for the pure-neural baseline (RQ2), JSD = 0.012 (RQ3), and the ablation study isolates the symbolic verification loop as the primary structural driver and the convex optimizer as the primary probabilistic calibration driver (RQ4). The fault-detection proxy analysis (Section VI-D) establishes that these gains translate into a 35.7-percentage-point increase in fault-revealing path coverage—consistent with the 25–40% improvement reported in empirical MBT deployment studies [8].

A. System-Under-Test Reference Architecture

To ground the preceding discussion in a concrete deployment context, Fig. 8 illustrates a representative System-Under-Test (SUT) architecture for an autonomous Cyber-Physical System (CPS). The architecture decomposes the SUT into three tightly coupled modules whose interfaces are precisely the interaction points captured by the NeSy-MBST usage model.

As systems of this kind become increasingly complex, the neuro-symbolic approach is particularly valuable for verifying safety-critical, autonomous, and learning-enabled CPS such as autonomous vehicles and unmanned aerial vehicles (UAVs) [47].

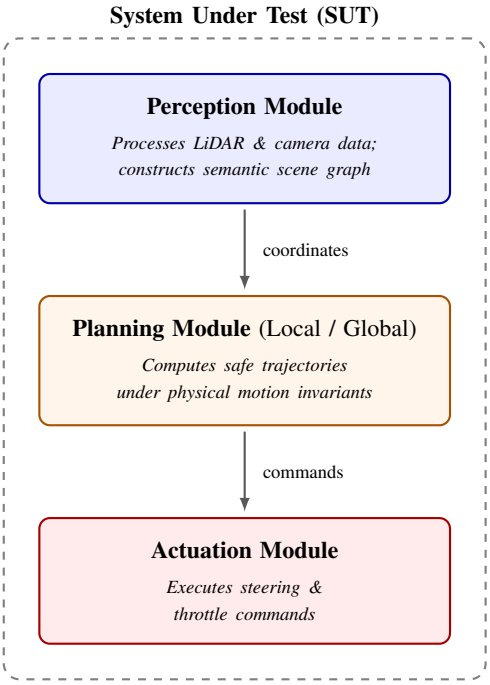

Fig. 8. Reference architecture for an autonomous CPS System-Under-Test. The three modules—Perception, Planning, and Actuation—correspond to distinct abstraction tiers whose interfaces are modelled as states and transitions in the NeSy-MBST usage model.

B. Adoption Guidelines

The following numbered guidelines synthesise the empirical findings into actionable recommendations for practitioners and researchers adopting neuro-symbolic MBST pipelines. Each guideline identifies the fault-detection mechanism it addresses.

1) **Automate specification mining with grammar-constrained oracles.** Process natural-language requirements through a grammar-constrained LLM oracle (output restricted to {`Yes`, `No`, `Unsure`}) to extract candidate states, transitions, and guards. Couple extraction with formal schema validation to prevent hallucinated artefacts entering the usage model [30], [48]. *Fault-detection mechanism:* eliminates structurally missing transitions that would otherwise blind the test suite to the fault-revealing paths they represent.
2) **Enforce a symbolic verification loop before probability assignment.** Validate every extracted transition against a symbolic feasibility memory before the usage model is parameterised. The ablation study demonstrates that this step alone accounts for a 35.7-percentage-point increase in transition coverage and a 93% reduction in JSD relative to unverified extraction [27], [44]. *Fault-detection mechanism:* ensures the model topology is structurally complete, the necessary precondition for transition-coverage-based fault detection.
3) **Calibrate transition probabilities via convex optimization, not direct LLM output.** Use an entropy-maximizing SLSQP solver to assign transition probabilities under the row-stochastic and domain constraints extracted from requirements. LLMs demonstrably produce miscalibrated numerical outputs [39]; solver-assigned probabilities achieve JSD = 0.012 vs. 0.157 for the uncalibrated baseline. *Fault-detection mechanism:* preserves

operationally weighted test allocation, ensuring that test effort is concentrated on the paths that real users actually exercise.

4) **Implement hierarchical multi-tier usage models for large systems.** Decompose the SUT into a high-level inter-module Markov chain and per-module subordinate chains. This controls state-space explosion while preserving the path-dependent behavioural patterns that drive reliability estimation [1], [35]. *Fault-detection mechanism:* prevents the combinatorial growth that causes purely flat models to omit rare but safety-critical fault-revealing paths.
5) **Deploy telemetry-driven closed-loop recalibration.** Feed runtime execution logs back into the NeSy-MBST pipeline to detect and correct model drift. The closed-loop adapter continuously updates transition probabilities whose empirical frequency deviates from the model prediction, maintaining alignment with the evolving operational profile [47], [49]. *Fault-detection mechanism:* prevents the model from becoming stale as usage patterns shift, which would otherwise cause test effort to be misdirected toward obsolete paths.

C. Recommendation for the Field: AI-Augmented Model Generation

Beyond the specific NeSy-MBST implementation, this paper's findings support a broader recommendation for the software-testing field: LLM augmentation of formal test-model generation should be treated as a standard engineering practice, not a research curiosity, provided two conditions hold. First, the LLM must operate under structural constraints (grammar-constrained output, symbolic verification loop) that prevent hallucination from corrupting the model topology. Second, numerical model parameters must be assigned by a certified solver, not by the LLM directly. When both conditions are met, the present results demonstrate that the manual modelling bottleneck can be eliminated without sacrificing the fault-detection properties that make MBST valuable.

What remains open is not whether the approach works, but *under what conditions it generalises*: the current evaluation covers two domains (autonomous vehicle CPS and e-commerce), two LLM backends (GPT-4o baselines, GPT-4.1-mini oracle), and models up to 42 states. The following open problems bound the scope of the current contribution:

- scalability to models with hundreds of states and concurrent multi-module interactions;
- performance on domains with ambiguous or informally-written requirements (e.g., agile user stories, verbal specifications);
- sensitivity to LLM provider, version, and temperature settings on oracle resolution rate and downstream F1; and
- the relationship between the proxy coverage metrics reported here and directly measured defect-detection rates in fault-seeding experiments.

D. Future Work: Industrial Case Studies

The most important next step for this research programme is **industrial validation via fault-seeding case studies**. The present evaluation establishes that NeSy-MBST-generated models achieve structural and probabilistic properties consistent with high fault-detection effectiveness, but does not directly measure defect-detection rates against a seeded or historical fault corpus. Future work should:

1) **Conduct fault-seeding experiments** on at least one medium-scale industrial SUT, comparing defect-detection rates for NeSy-MBST-generated test suites against (a) manually modelled MBST suites and (b) unstructured script-based regression suites. The transition-coverage and JSD proxies from the present study should be validated against these direct measurements.
2) **Evaluate generalisability across domains and requirement styles.** Replicate the evaluation on domains beyond CPS and e-commerce—telecommunications protocols, medical device workflows, and automotive functional safety (ISO 26262)—to establish the boundary conditions for the approach.
3) **Assess multi-annotator agreement** on ground-truth usage model construction, replacing the single-author annotation used in this study with an inter-rater reliability protocol.
4) **Investigate the three-valued oracle at scale.** Characterise how escalation rate, oracle temperature, and requirement ambiguity interact to affect convergence speed and model completeness on larger benchmarks.

E. Concluding Remarks

By enforcing structural coherence through the L^* algorithm, probabilistic calibration through symbolic solvers, and empirical fidelity through telemetry-driven recalibration, NeSy-MBST produces test pipelines that are *structurally complete*, *operationally calibrated*, and *capable of certifying system reliability* at the rigour demanded by safety-critical CPS [1], [49]. The framework removes the long-standing manual modelling bottleneck that has confined rigorous statistical testing to specialist teams, making it accessible to broader software development practice. As autonomous and learning-enabled systems proliferate, we anticipate that neuro-symbolic integration of this kind will transition from a research contribution to an engineering standard—backed by the empirical fault-detection validation that future industrial case studies will supply.

Replication Package

The NeSy-MBST implementation, ablation study, and figure-generation scripts are available at https://github.com/nathangtg/llm-mbst-research. All results are reproducible with `python run_ablation.py` (fixed seed 42) using the Azure OpenAI backend configured in `nesy_mbst/.env`.

References

[1] M. Utting, A. Pretschner, and B. Legeard, "A taxonomy of model-based testing approaches," *Software Testing, Verification and Reliability*, vol. 22, no. 5, pp. 297–312, 2012.

[2] J. A. Whittaker and J. H. Poore, "Markov analysis of software specifications," *ACM Transactions on Software Engineering and Methodology*, vol. 2, no. 1, pp. 93–106, 1993.

[3] J. A. Whittaker and M. G. Thomason, "A Markov chain model for statistical software testing," *IEEE Transactions on Software Engineering*, vol. 20, no. 10, pp. 812–824, 1994.

[4] S. Abdulkarim, E. Boyd, K. Bridi, A. Tufenkjian, B. Chen, and G. Mussbacher, "Structure- and event-driven frameworks for state machine modeling with large language models," in *arXiv preprint arXiv:2604.00275*, 2026, arXiv:2604.00275.

[5] I. S. W. B. Prasetya and R. Klomp, "Test model coverage analysis under uncertainty," *Software and Systems Modeling*, vol. 20, no. 2, pp. 383–403, 2021.

[6] E. Alégroth, K. Karl, H. Rosshagen, T. Helmfridsson, and N. Olsson, "Practitioners' best practices to adopt, use or abandon model-based testing with graphical models for software-intensive systems," *Empirical Software Engineering*, 2022.

[7] F. C. Ferrari, V. H. S. Durelli, S. F. Andler, J. Offutt, M. Saadatmand, and N. Müllner, "On transforming model-based tests into code: A systematic literature review," *Software Testing, Verification and Reliability*, 2023.

[8] V. Garousi, A. B. Keleş, Y. Balaman, Z. Ö. Güler, and A. Arcuri, "Model-based testing in practice: An experience report from the web applications domain," *Journal of Systems and Software*, vol. 180, p. 111032, 2021.

[9] Z. Li, X. Wu, D. Zhu, M. Cheng, S. Chen, F. Zhang, X. Xie, L. Ma, and J. Zhao, "Generative model-based testing on decision-making policies," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2023.

[10] V. Garousi, A. B. Keleş, Y. Balaman, A. Mermer, and Z. Ö. Güler, "Coverage measurement in model-based testing of web applications: Tool support and an industrial experience report," in *2024 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, 2024.

[11] Y.-J. Shin, E. Cho, and D.-H. Bae, "PASTA: An efficient proactive adaptation approach based on statistical model checking for self-adaptive systems," in *Proceedings of the 24th International Conference on Fundamental Approaches to Software Engineering (FASE)*, ser. Lecture Notes in Computer Science, vol. 12649, 2021.

[12] G. Agha and K. Palmskog, "A survey of statistical model checking," *ACM Transactions on Modeling and Computer Simulation*, vol. 28, no. 1, pp. 6:1–6:39, 2018.

[13] A. Legay, B. Delahaye, and S. Bensalem, "Statistical model checking: An overview," in *Runtime Verification (RV 2010)*, ser. Lecture Notes in Computer Science, vol. 6418, 2010, pp. 122–135.

[14] K. G. Larsen and A. Legay, "Statistical model checking: Past, present, and future," in *Proceedings of the 6th International Symposium on Leveraging Applications of Formal Methods (ISoLA)*, ser. Lecture Notes in Computer Science, vol. 8803, 2014, pp. 135–142.

[15] M. Camilli, A. Gargantini, P. Scandurra, and C. Trubiani, "Uncertainty-aware exploration in model-based testing," in *Proceedings of the 14th IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2021.

[16] C. E. Budde, A. Hartmanns, T. Meggendorfer, M. Weininger, and P. Wienhöft, "Sound statistical model checking for probabilities and expected rewards," in *Proceedings of the 31st International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, ser. Lecture Notes in Computer Science, vol. 15696, 2025, pp. 167–190.

[17] S. Kanav, J. Křetínský, and K. G. Larsen, "Statistical model checking the 2024 edition!" in *Bridging the Gap Between AI and Reality*, ser. Lecture Notes in Computer Science, 2025, vol. 15217.

[18] R. Soltani, M. Volk, L. Diamonte, M. Lopuhaä-Zwakenberg, and M. Stoelinga, "Optimal spare management via statistical model checking: A case study in research reactors," in *Proceedings of the 28th International Conference on Formal Methods for Industrial Critical Systems (FMICS)*, ser. Lecture Notes in Computer Science, vol. 14290, 2023, pp. 205–223.

[19] M. Kwiatkowska, G. Norman, and D. Parker, "PRISM: Statistical model checking," https://www.prismmodelchecker.org/manual/RunningPRISM/StatisticalModelChecking, 2024, university of Oxford; see also *CAV* 2011, LNCS 6806, pp. 585–591.

[20] P. Vasconcelos, A. Manocha, A. Levisse, and D. Atienza, "Rigorous evaluation of computer processors with statistical model checking," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023.

[21] Y. Liu and A. Fang, "PRG4CNN: A probabilistic model checking-driven robustness guarantee framework for CNNs," *Entropy*, vol. 27, no. 2, p. 163, 2025.

[22] B. K. Aichernig and R. A. Schumi, "Statistical model checking meets property-based testing," in *Proceedings of the 10th IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 2017, pp. 390–400.

[23] M. Gerhold and M. Stoelinga, "Model-based testing of probabilistic systems," in *Proceedings of the 19th International Conference on Fundamental Approaches to Software Engineering (FASE)*, ser. Lecture Notes in Computer Science, vol. 9633, 2016, pp. 251–268.

[24] R. Lassaigne and S. Peyronnet, "Approximate planning and verification for large Markov decision processes," *International Journal on Software Tools for Technology Transfer*, 2014.

[25] M. Utting and B. Legeard, *Practical Model-Based Testing: A Tools Approach*. Morgan Kaufmann, 2007.

[26] S. J. Prowell, "Using Markov chain usage models to test complex systems," in *Proceedings of the 37th Annual Hawaii International Conference on System Sciences (HICSS)*, 2004.

[27] F. Böhr, "A constraint-based approach to the representation of software usage models," *Journal of Systems and Software*, vol. 86, no. 4, pp. 1064–1080, 2013.

[28] A. Pretschner, "Model-based testing in practice," in *FM 2005: Formal Methods*, ser. Lecture Notes in Computer Science, vol. 3472. Springer, 2005, pp. 537–541.

[29] H. Wei, L. Chen, Z. Du, Y. Wu, H. Huang, Y. Liu, G. Cheng, F. Xu, L. Wang, and B. Mao, "Unleashing the power of LLM to infer state machine from the protocol implementation," in *arXiv preprint arXiv:2405.00393*, 2024, arXiv:2405.00393.

[30] C. Huang, D. Wang, and Z. Q. Zhou, "LLM-assisted model-based fuzzing of protocol implementations," in *arXiv preprint arXiv:2508.01750*, 2025, arXiv:2508.01750.

[31] J. Thomasson, "Benchmarking large language models in UML diagram generation from informal notations," Master's thesis, Linköping University, 2024, diVA:diva2:1983182.

[32] K. Chen, Y. Li, C. Guo, X. Che, S. Yu, C. Zhang, and D. Hao, "GLIB: Towards automated test oracle for graphically-rich applications," in *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2021.

[33] Eggplant, "Using AI to find elements by description," https://docs.eggplantsoftware.com/epf/epf-find-by-description/, 2024, vendor documentation, accessed 2026.

[34] X. Guo, C. Li, and T. Tsuchiya, "Boundary value test input generation using prompt engineering with LLMs: Fault detection and coverage analysis," in *arXiv preprint arXiv:2501.14465*, 2025, arXiv:2501.14465.

[35] G. V. Bochmann and A. Petrenko, "Improved usage model for web application reliability testing," *Journal of Systems and Software*, 2011.

[36] National Research Council, "Statistics, testing, and defense acquisition: Background papers," in *Statistics, Testing, and Defense Acquisition*. National Academies Press, 1998.

[37] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, arXiv:1706.03762.

[38] S. Kang, J. Yoon, and S. Yoo, "Large language models are few-shot testers: Exploring LLM-based general bug reproduction," in *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2023, pp. 2312–2323.

[39] S. Frieder, L. Pinchetti, A. Chevalier, R.-R. Griffiths, T. Salvatori, T. Lukasiewicz, P. Petersen, and J. Berner, "Mathematical capabilities of ChatGPT," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.

[40] S. J. Prowell, "Computing system reliability using Markov chain usage models," *Journal of Systems and Software*, vol. 73, no. 3, pp. 219–225, 2004.

[41] ——, "Using Markov chain usage models to test complex systems," in *Proceedings of the 37th Annual Hawaii International Conference on System Sciences (HICSS)*, 2004.

[42] B. Wen, R. Zhang, Y. Chen, H. Xie, and L.-Z. Guo, "Aligning progress and feasibility: A neuro-symbolic dual memory framework for long-horizon LLM agents," *arXiv preprint arXiv:2604.02734*, 2026, arXiv:2604.02734.

[43] A. d'Avila Garcez and L. C. Lamb, "Neurosymbolic AI: The 3rd wave," *Artificial Intelligence Review*, 2023.

[44] D. Angluin, "Learning regular sets from queries and counterexamples," *Information and Computation*, vol. 75, no. 2, pp. 87–106, 1987.

[45] M. Vazquez-Chanlatte, K. Elmaaroufi, S. J. Witwicki, M. Zaharia, and S. A. Seshia, "L^*LM: Learning automata from examples using natural language oracles," *arXiv preprint arXiv:2402.07051*, 2024, arXiv:2402.07051.

[46] M. R. A. Praja, Riskiana, and D. S. Kusumo, "Software testing on E-Commerce application using model-based testing Markov chain," in *2024 2nd International Conference on Software Engineering and Information Technology (ICoSEIT)*, 2024, pp. 287–292.

[47] X. Zheng *et al.*, "LLM-guided synthesis, testing, and verification of learning-enabled cyber-physical systems," NII Shonan Meeting, Tech. Rep. 235, 2025, available: https://shonan.nii.ac.jp/docs/No.235.pdf.

[48] A. Beg, D. O'Donoghue, and R. Monahan, "Formalising software requirements with large language models," *arXiv preprint arXiv:2506.14627*, 2025, arXiv:2506.14627.

[49] X. Zheng, "NeuroStrata: Harnessing neuro-symbolic paradigms for improved testability and verifiability of autonomous CPS," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2023.